
Data Structures

BS (CS) _Fall_2024

Lab_08 Tasks



Learning Objectives:

1. Queue

Lab Tasks

Submission Instructions

1. Create a separate **.h** and **.cpp** file for each class.
2. Now create a new folder with name *ROLLNO_SEC_LAB07* e.g. **i23XXXX_A_LAB07**(Also name your **cpp** file like this.)
3. Move all of your files to this newly created directory and compress it into a **.zip file**. Return the test file as well.
4. Now you have to submit this zipped file on Google Classroom.
5. If you don't follow the above-mentioned submission instruction, you will be marked **zero**.
6. Plagiarism or AI generated code in the Lab Task will result in **zero** marks in the whole category.

Question 1

Multi-Level Feedback Queue Scheduler (Node-Based

Queue)Objective:

Design and implement a multi-level feedback queue scheduler in C++ using node-based queues. This scheduler will manage multiple queues with different priorities and use the First-Come, First-

Served (FCFS) scheduling algorithm within each queue. The task will involve creating classes and functions to simulate the scheduling process.

Task Description:

Process Class:

- Create a Process class to represent individual processes. Each process should have the following attributes:
 - Process ID
 - Priority (0 = high, 1 = medium, 2 = low)
 - Burst Time (the time it takes to execute the process)
 - Next pointer (for linking processes in the queue)

CustomQueue Class:

- Implement a CustomQueue class for node-based queues. Each queue node should contain a Process object and a next pointer to the next node.
- Include the following member functions in the CustomQueue class:
 - isEmpty(): Check if the queue is empty.
 - enqueue(Process* process): Add a process to the queue.
 - dequeue(): Remove and return the process at the front of the queue.

MultiLevelFeedbackQueueScheduler Class:

- Create a MultiLevelFeedbackQueueScheduler class to manage the scheduling process.
- Include the following member functions in the MultiLevelFeedbackQueueScheduler class:
 - addProcess(Process process): Add a process to the appropriate priority queue based on its priority.
 - executeProcesses(): Implement the scheduling logic using FCFS within each priority queue and promote processes after a certain number of executions.

Question 2

Round robin is a scheduling algorithm that an operating system uses to time share computational resources of a processor between tasks. Each task is given a specific time slot (**Quantum**) to execute on a processor (**CPU Time**), once this time slot expires and the task is not yet

completed, it is preempted (**dequeue**) and added to the back of the queue (**enqueue**) with its **RemainingExecution Time**. Then the next task (**front**) in the queue is selected and this process continues until all tasks have finished execution.



Your task is to simulate this process using a queue. Implement a function `roundRobin()` that takes input a Queue of execution time and a time quantum. Then it simulates the process of task execution until the queue gets empty. Example of output is displayed below.

```
Execution Time: 50
Remaining Time: 20
Task is not completed, it is being re-scheduled

Execution Time: 30
Remaining Time: 0
Task is completed, it is removed from queue

Execution Time: 80
Remaining Time: 50
Task is not completed, it is being re-scheduled

Execution Time: 100
Remaining Time: 70
Task is not completed, it is being re-scheduled

Execution Time: 20
Remaining Time: -10
Task is completed, it is removed from queue

Execution Time: 50
Remaining Time: 20
Task is not completed, it is being re-scheduled

Execution Time: 70
Remaining Time: 40
Task is not completed, it is being re-scheduled

Execution Time: 20
Remaining Time: -10
Task is completed, it is removed from queue

Execution Time: 40
Remaining Time: 10
Task is not completed, it is being re-scheduled

Execution Time: 10
Remaining Time: -20
Task is completed, it is removed from queue
```